CS 255: Computer Architecture and Machine Level Programming

Prof. Martha Barker

MIPS Data Manipulation

MIPS Data Lives in Three Places

1. In a register
2. In the instruction as an immediate (16 bit constant embedded in the instruction)
3. In memory (2^{32} byte capacity)

Registers

MIPS Registers

MIPS has 32 registers, each 32 bits wide.

Each register is identified by $<name>.

Physically they are all the same except $zero, however their use is governed by convention.

MIPS Registers

MIPS has 32 registers, each 32 bits wide.

Each register is identified by $<name>.

Physically they are all the same except $zero, however their use is governed by convention.

RISC Design Principle #3: Smaller is Faster

- MIPS only has a small number of registers

General Purpose Registers

MIPS has 18 general purpose registers: $t0-$t9 and $s0-$s7

Temporary registers ($t0-$t9) are used for short-term values that are *not* preserved across function calls.

Saved Registers ($s0-$s7) are used for values that *are* preserved across function calls.

General Purpose Registers

MIPS has 18 general purpose registers: $t0-$t9 and $s0-$s7

Temporary registers ($t0-$t9) are used for short-term values that are *not* preserved across function calls.

Use $t registers for scratch work

Saved Registers ($s0-$s7) are used for values that *are* preserved across function calls.

Use $s registers for variables

Function Call Registers

MIPS has 7 registers for function calls: $a0-$a3, $v0-$v1, and $ra

Argument registers ($a0-$a3) are used to pass function parameters.

Result registers ($v0-$v1) are used to return function results.

The return address register ($ra) is used to return to the point of origin when the function is complete.

Special Purpose Registers

$zero always holds a constant zero value (cannot be overwritten).

MIPS has three pointer registers: $gp, $sp, and $fp that are used to manage memory.

The stack pointer ($sp) points to the most recently allocated memory address in the stack.

The frame pointer ($fp) points to the base of a function's stack frame.

The global pointer ($gp) points to where global variables are stored.

Reserved Registers

MIPS has three reserved registers: $at and $k0-$k1

The assembler temporary register ($at) is reserved for the assembler to expand pseudo-instructions.

Kernel registers ($k0-$k1) are reserved for the operating system to handle exceptions and interrupts.

★ DO NOT USE RESERVED REGISTERS

MIPS Registers

Name	Number	Use
$zero	0	constant zero
$at	1	reserved for assembler
$v0-$v1	2-3	function result(s)
$a0-$a3	4-7	function arguments(s)
$t0-$t7	8-15	temporaries
$s0-$s7	16-23	saved temporaries
$t8-$t9	24-25	temporaries
$k0-$k1	26-27	reserved for OS
$gp	28	global pointer
$sp	29	stack pointer
$fp	30	frame pointer
$ra	31	return address

Immediates

Immediate Operands

An *immediate* is a small constant embedded in the instruction.

- 16 bit field
- Value can range from -32,768 to 32,767

Small constants are common, so immediate operands spare a register.

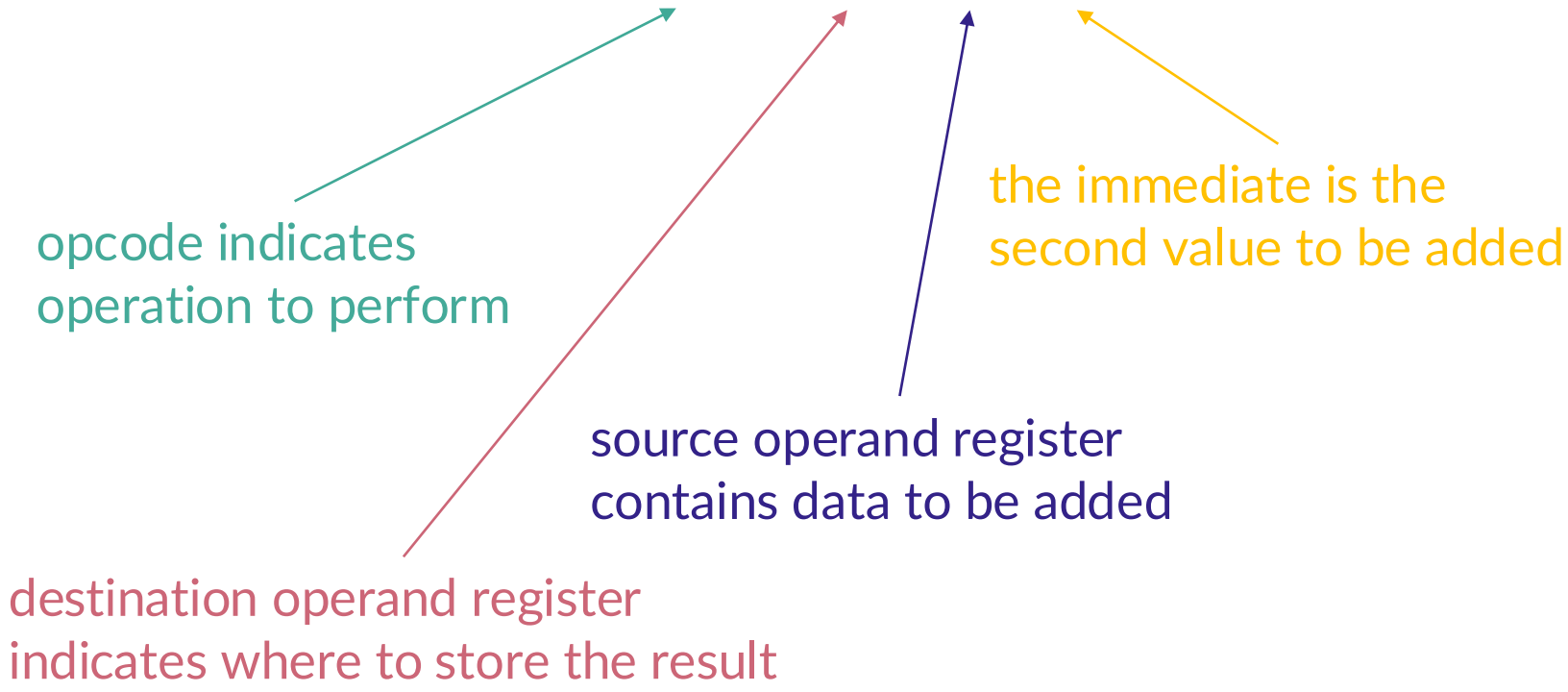
Add Immediate in MIPS

C code:

```
a = b + 1;
```

MIPS assembly code:

```
addi $a, $b, 1
```



Immediate Computational Instructions

operation $rt, $rs, imm

"perform operation on the contents of register $rs and the 16 bit immediate imm and put the result in $rt"



MIPS Reference Data ①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	0 / 20_{hex}
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		0 / 21_{hex}
And	and	R	R[rd] = R[rs] & R[rt]		0 / 24_{hex}
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5_{hex}
Jump	j	J	PC=JumpAddr	(5)	2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5)	3_{hex}
Jump Register	jr	R	PC=R[rs]		0 / 08_{hex}
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs]+SignExtImm](7:0)}	(2)	24_{hex}
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs]+SignExtImm](15:0)}	(2)	25_{hex}
Load Linked	ll	I	R[rt] = M[R[rs]+SignExtImm]	(2,7)	30_{hex}
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}		f_{hex}
Load Word	lw	I	R[rt] = M[R[rs]+SignExtImm]	(2)	23_{hex}
Nor	nor	R	R[rd] = ~ (R[rs] | R[rt])		0 / 27_{hex}
Or	or	R	R[rd] = R[rs] | R[rt]		0 / 25_{hex}
Or Immediate	ori	I	R[rt] = R[rs] | ZeroExtImm	(3)	d_{hex}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0		0 / $2a_{hex}$
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < SignExtImm)? 1 : 0	(2)	a_{hex}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0	(2,6)	b_{hex}
Set Less Than Unsig.	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6)	0 / $2b_{hex}$
Shift Left Logical	sll	R	R[rd] = R[rt] << shamt		0 / 00_{hex}
Shift Right Logical	srl	R	R[rd] = R[rt] >> shamt		0 / 02_{hex}
Store Byte	sb	I	M[R[rs]+SignExtImm](7:0) = R[rt](7:0)	(2)	28_{hex}
Store Conditional	sc	I	M[R[rs]+SignExtImm] = R[rt]; R[rt] = (atomic) ? 1 : 0	(2,7)	38_{hex}
Store Halfword	sh	I	M[R[rs]+SignExtImm](15:0) = R[rt](15:0)	(2)	29_{hex}
Store Word	sw	I	M[R[rs]+SignExtImm] = R[rt]	(2)	$2b_{hex}$
Subtract	sub	R	R[rd] = R[rs] - R[rt]	(1)	0 / 22_{hex}
Subtract Unsigned	subu	R	R[rd] = R[rs] - R[rt]		0 / 23_{hex}

32-Bit Constants

Larger constants do not fit in the 16-bit immediate field, so they must be loaded into a register before use.

Load a 32-bit number into a register:



32-Bit Constants

Larger constants do not fit in the 16-bit immediate field, so they must be loaded into a register before use.

Load a 32-bit number into a register:



1. Set the top 16 bits:

```
lui $t1, 0xCAFE
```



"load upper immediate"

32-Bit Constants

Larger constants do not fit in the 16-bit immediate field, so they must be loaded into a register before use.

Load a 32-bit number into a register:



1. Set the top 16 bits:

```
lui $t1, 0xCAFE
```



"load upper immediate"

2. Set the bottom 16 bits:

```
ori $t1, $t1, 0x0B0E
```



Pseudo-Instructions

Assemblers offer pseudo-instructions for programmer convenience.

- Not part of the ISA
- Not a real instruction (removed before becoming machine code)
- Assembler translates pseudo-instruction into short sequence of real instructions

"load immediate" loads a 32-bit value into a register

```
li $t1, 0xCAFE0B0E
```



Data Pseudo-Instructions

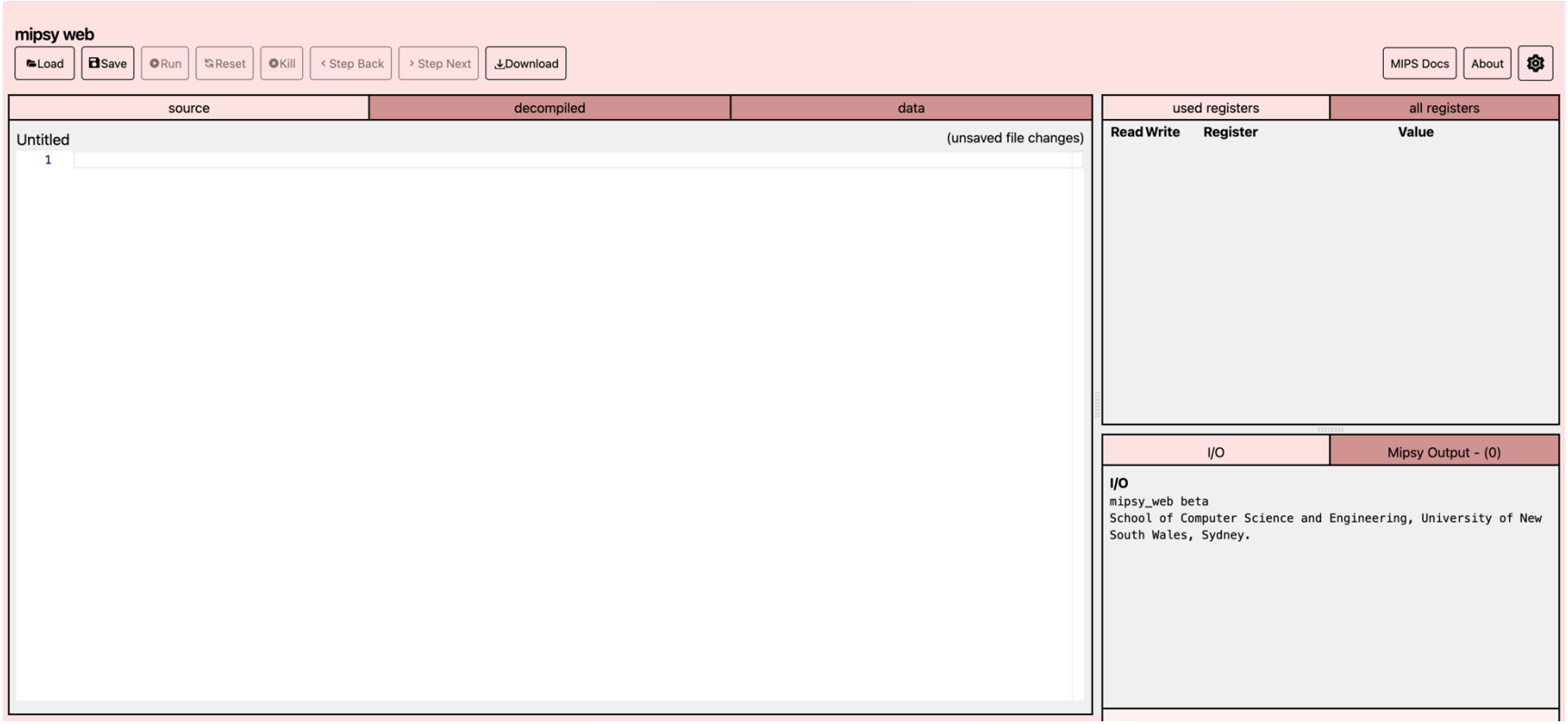
Action	Pseudo-Instruction	Translation
load immediate $0 \le imm \le 65535$	li $rd, imm	ori $rd, $0, imm
load immediate $-32768 \le imm < 0$	li $rd, imm	addiu $rd, $0, imm
load immediate	li $rd, imm	lui $rd, hi16(imm) ori $rd, $rd, lo16(imm)
move	move $rd, $rs	or $rd, $rs, $0

Arithmetic Pseudo-Instructions

Action	Pseudo-Instruction	Translation
multiply	mul $rd, $rs, $rt	mult $rs, $rt mflo $rd
negate unsigned	negu $rd, $rs	subu $rd, $0, $rs
set if equal	seq $rd, $rs, $rt	xor $rd, $rs, $rt sltiu $rd, $rd, 1
set if greater than or equal	sge $rd, $rs, $rt	slt $rd, $rs, $rt xori $rd, $rd, 1
set if greater than or equal unsigned	sgeu $rd, $rs, $rt	sltu $rd, $rs, $rt xori $rd, $rd, 1
set if greater than	sgt $rd, $rs, $rt	slt $rd, $rt, $rs

Simulating MIPS Code

We will use mipsy: https://cgi.cse.unsw.edu.au/~cs1521/mipsy/

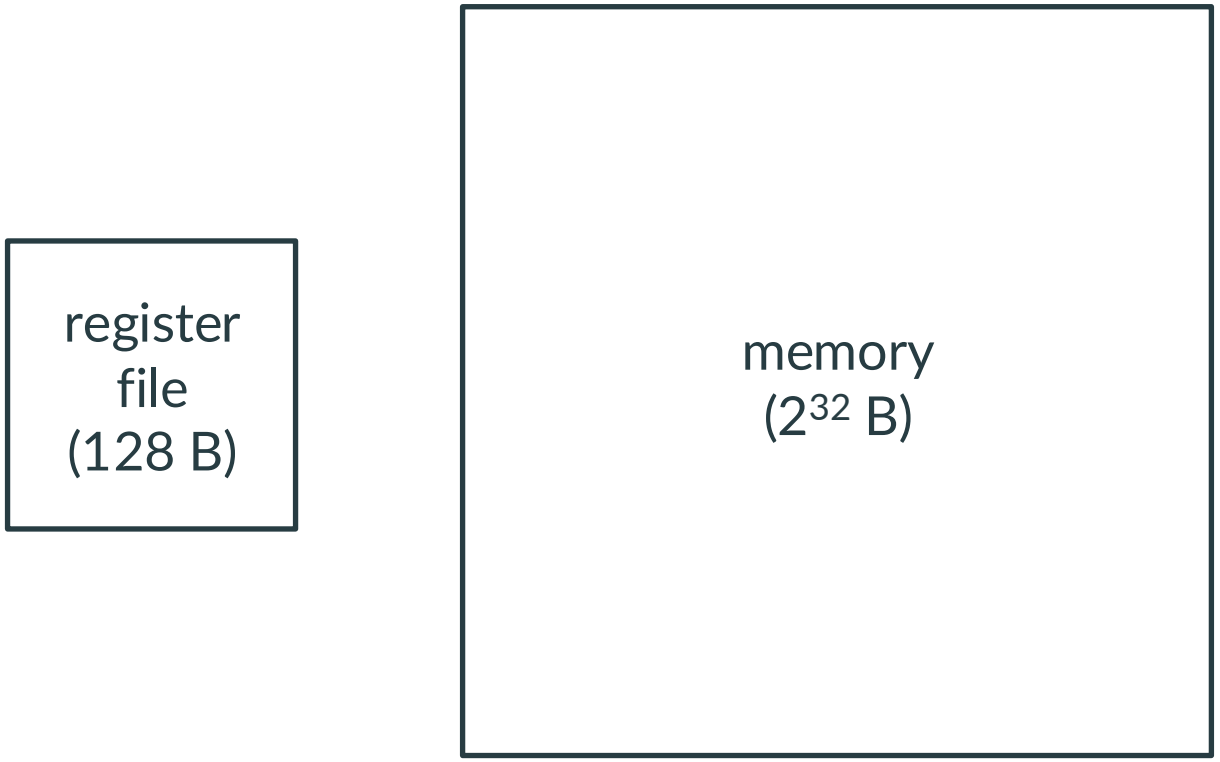


Demo: mipsy

Memory

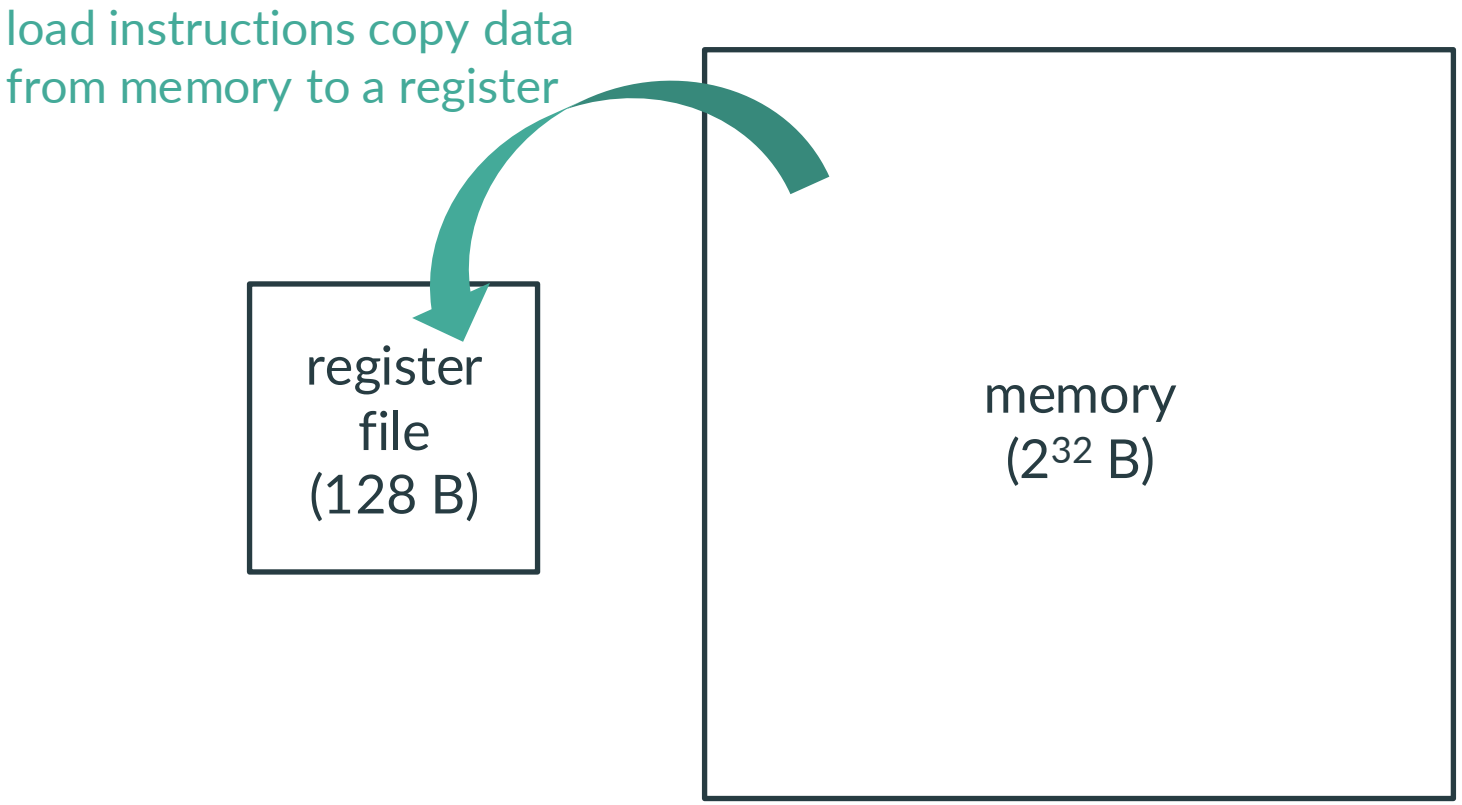
Computing with Data in Memory

Most programs use more data than can fit in registers, so we use memory.



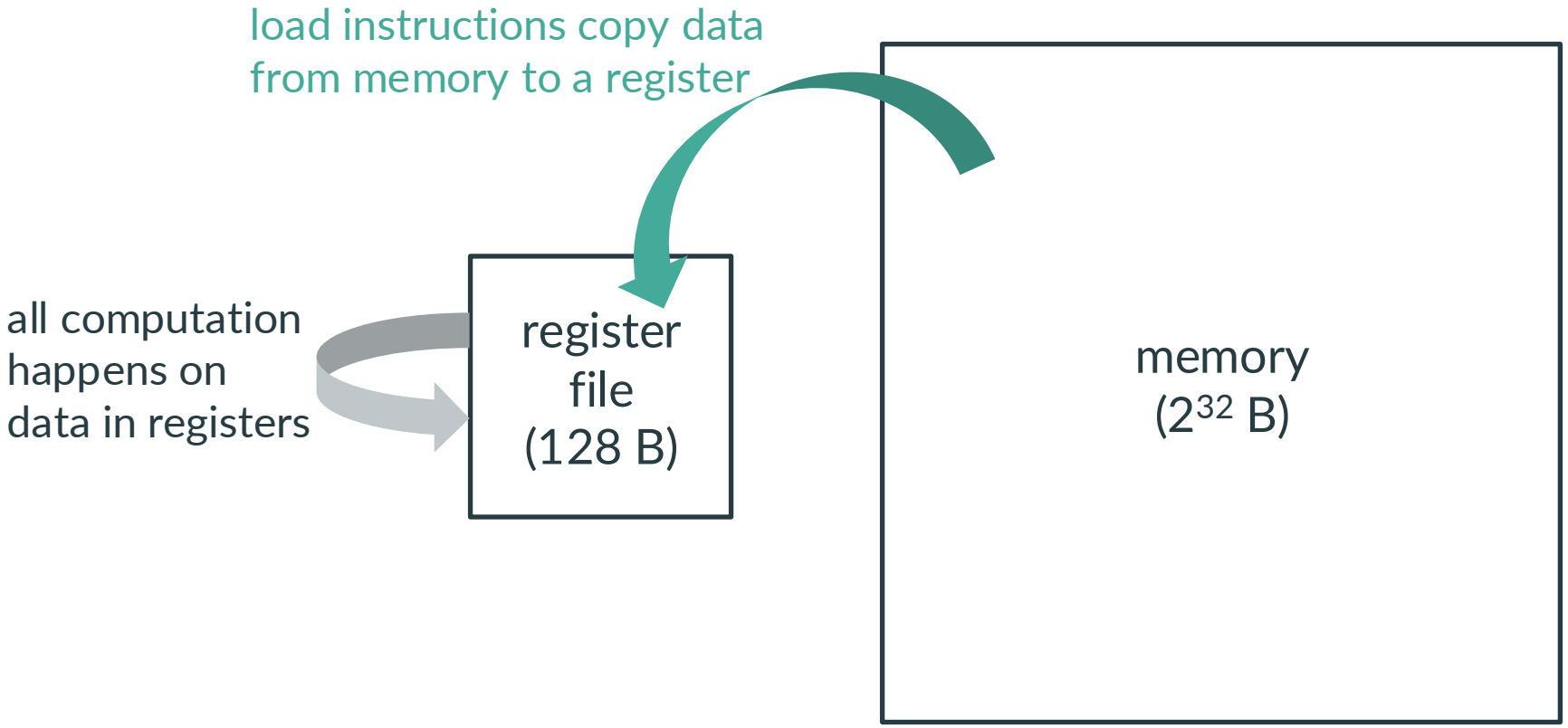
Computing with Data in Memory

Most programs use more data than can fit in registers, so we use memory.



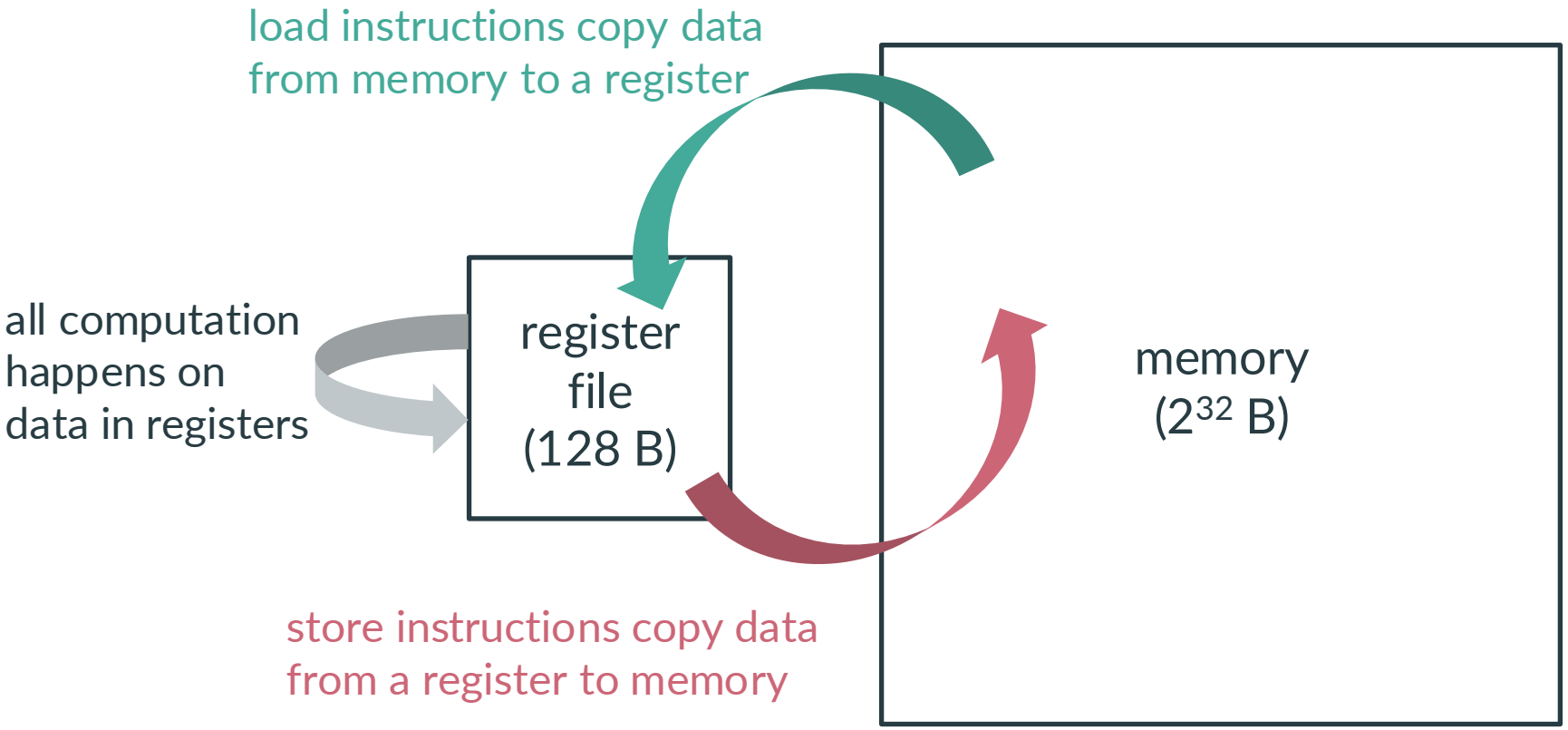
Computing with Data in Memory

Most programs use more data than can fit in registers, so we use memory.



Computing with Data in Memory

Most programs use more data than can fit in registers, so we use memory.



Base Addressing in MIPS

MIPS expresses addresses as a base plus an offset.

address = base pointer + offset

stored in a register immediate

Load a Word of Data

MIPS assembly code:

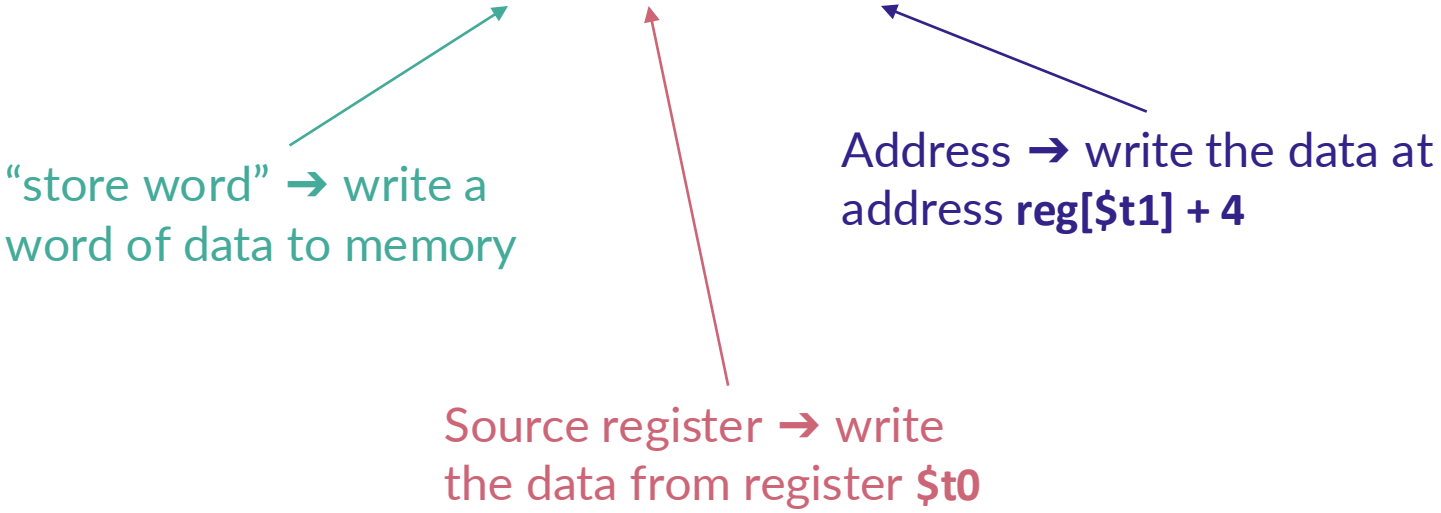
```
lw $t0, 4($t1)
```



Store a Word of Data

MIPS assembly code:

```
sw $t0, 4($t1)
```



Base Addressing is Versatile

Base addressing lets us keep one pointer to the start of a structure, then access fields using fixed offsets relative to the base.

```
struct Node {
  int value;
  struct Node* next;
};

struct Node n;
```

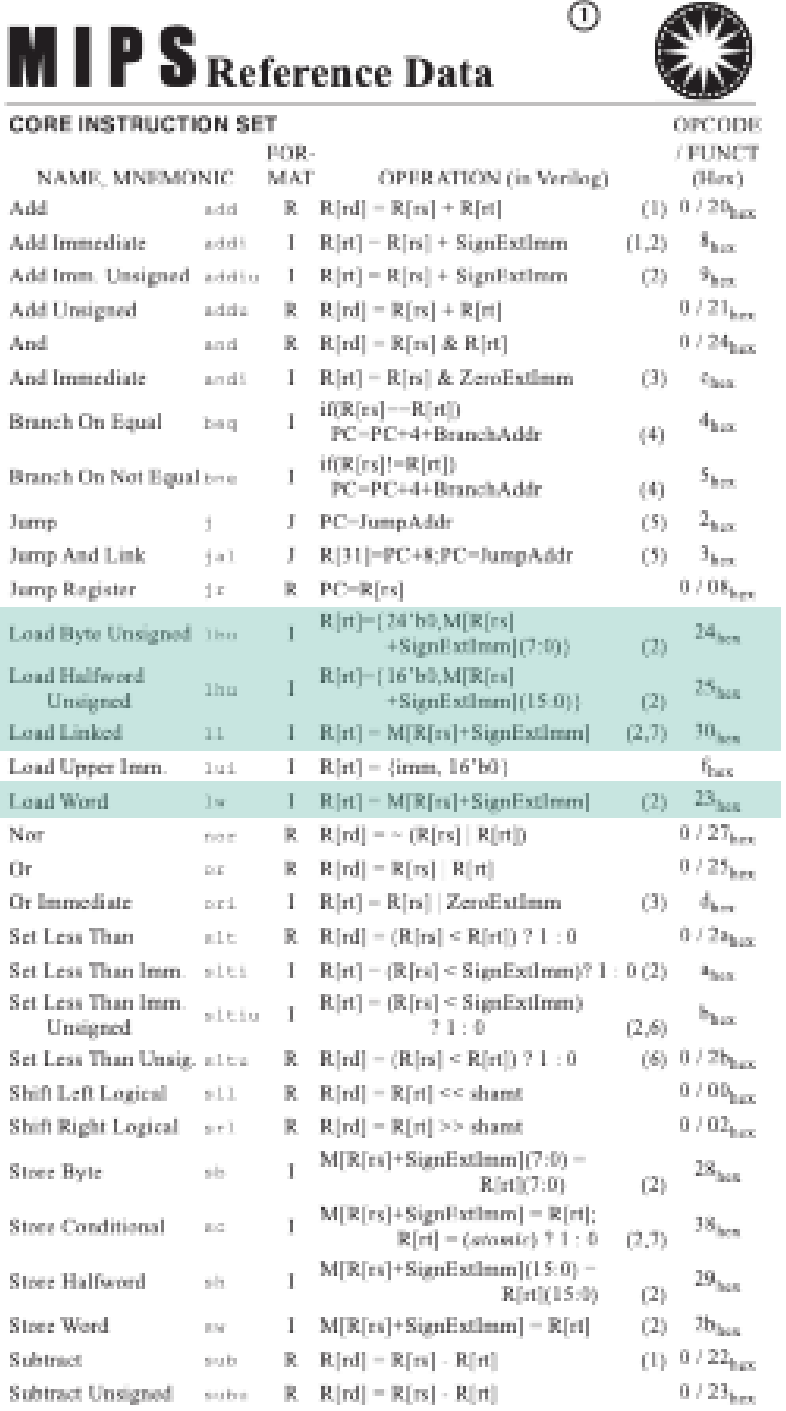
```
# if $s0 points to n
lw $t0, 0($s0) # load n.value
lw $t1, 4($s0) # load n.next
```

Load Instructions

l[w|bu|hu] $rt, imm($rs)

“load [word|byte|halfword] from memory address reg[$rs]+imm and put the data in $rt”

*load linked outside CS 255 scope



MIPS Reference Data ①

CORE INSTRUCTION SET

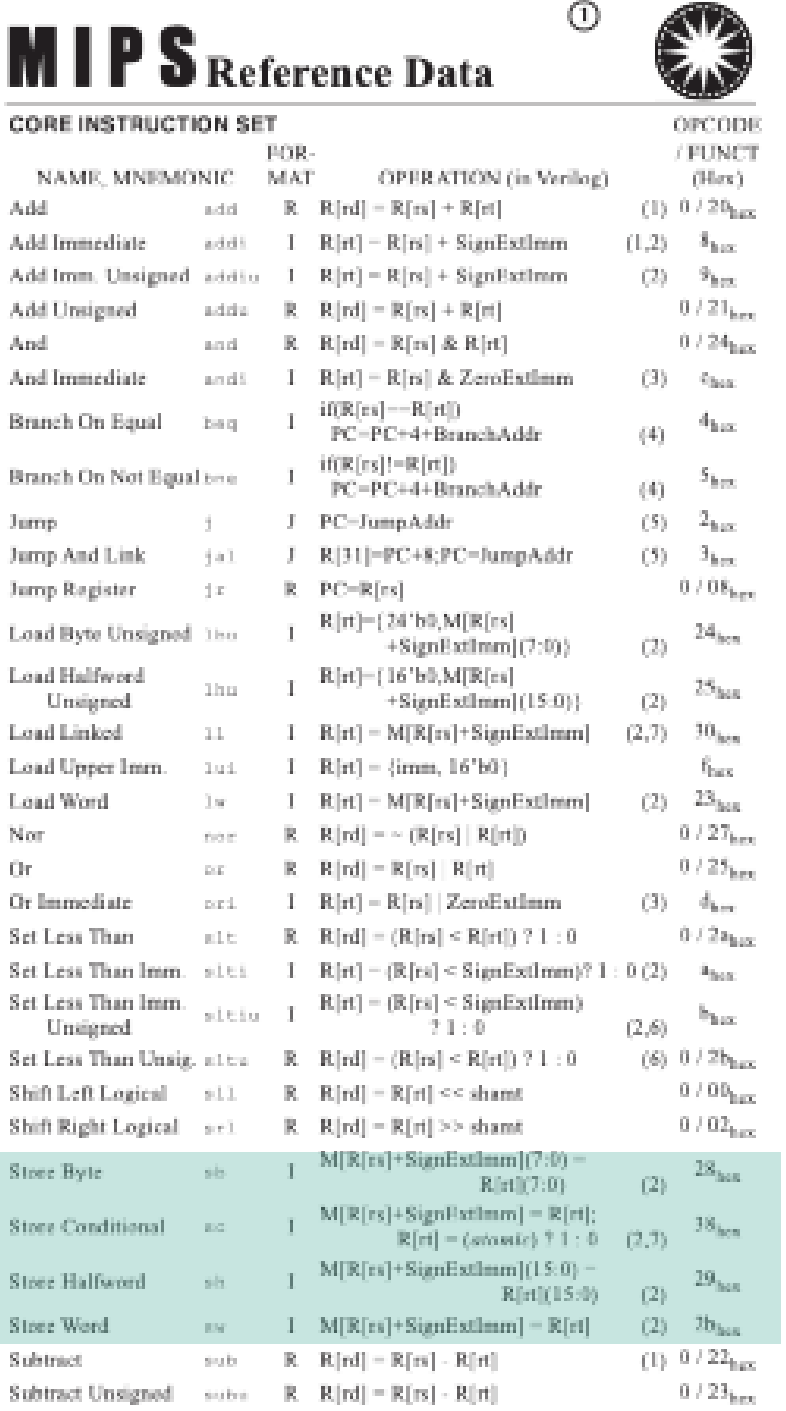
NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	0 / 20_{hex}
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		0 / 21_{hex}
And	and	R	R[rd] = R[rs] & R[rt]		0 / 24_{hex}
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5_{hex}
Jump	j	J	PC=JumpAddr	(5)	2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5)	3_{hex}
Jump Register	jr	R	PC=R[rs]		0 / 08_{hex}
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs]+SignExtImm](7:0)}	(2)	24_{hex}
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs]+SignExtImm](15:0)}	(2)	25_{hex}
Load Linked	ll	I	R[rt] = M[R[rs]+SignExtImm]	(2,7)	30_{hex}
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}		f_{hex}
Load Word	lw	I	R[rt] = M[R[rs]+SignExtImm]	(2)	23_{hex}
Nor	nor	R	R[rd] = ~ (R[rs] | R[rt])		0 / 27_{hex}
Or	or	R	R[rd] = R[rs] | R[rt]		0 / 25_{hex}
Or Immediate	ori	I	R[rt] = R[rs] | ZeroExtImm	(3)	d_{hex}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0		0 / $2a_{hex}$
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < SignExtImm)? 1 : 0	(2)	a_{hex}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0	(2,6)	b_{hex}
Set Less Than Unsig.	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6)	0 / $2b_{hex}$
Shift Left Logical	sll	R	R[rd] = R[rt] << shamt		0 / 00_{hex}
Shift Right Logical	srl	R	R[rd] = R[rt] >> shamt		0 / 02_{hex}
Store Byte	sb	I	M[R[rs]+SignExtImm](7:0) = R[rt](7:0)	(2)	28_{hex}
Store Conditional	sc	I	M[R[rs]+SignExtImm] = R[rt]; R[rt] = (atomic) ? 1 : 0	(2,7)	38_{hex}
Store Halfword	sh	I	M[R[rs]+SignExtImm](15:0) = R[rt](15:0)	(2)	29_{hex}
Store Word	sw	I	M[R[rs]+SignExtImm] = R[rt]	(2)	$2b_{hex}$
Subtract	sub	R	R[rd] = R[rs] - R[rt]	(1)	0 / 22_{hex}
Subtract Unsigned	subu	R	R[rd] = R[rs] - R[rt]		0 / 23_{hex}

Store Instructions

s[w|b|h] $rt, imm($rs)

“store [word|byte|halfword] of the contents of register $rt at memory address reg[$rs]+imm”

*store conditional outside CS 255 scope



MIPS Reference Data ①

CORE INSTRUCTION SET

NAME, MNEMONIC		FOR-MAT	OPERATION (in Verilog)		OPCODE / FUNCT (Hex)
Add	add	R	R[rd] = R[rs] + R[rt]	(1)	0 / 20_{hex}
Add Immediate	addi	I	R[rt] = R[rs] + SignExtImm	(1,2)	8_{hex}
Add Imm. Unsigned	addiu	I	R[rt] = R[rs] + SignExtImm	(2)	9_{hex}
Add Unsigned	addu	R	R[rd] = R[rs] + R[rt]		0 / 21_{hex}
And	and	R	R[rd] = R[rs] & R[rt]		0 / 24_{hex}
And Immediate	andi	I	R[rt] = R[rs] & ZeroExtImm	(3)	c_{hex}
Branch On Equal	beq	I	if(R[rs]==R[rt]) PC=PC+4+BranchAddr	(4)	4_{hex}
Branch On Not Equal	bne	I	if(R[rs]!=R[rt]) PC=PC+4+BranchAddr	(4)	5_{hex}
Jump	j	J	PC=JumpAddr	(5)	2_{hex}
Jump And Link	jal	J	R[31]=PC+8;PC=JumpAddr	(5)	3_{hex}
Jump Register	jr	R	PC=R[rs]		0 / 08_{hex}
Load Byte Unsigned	lbu	I	R[rt]={24'b0,M[R[rs]+SignExtImm](7:0)}	(2)	24_{hex}
Load Halfword Unsigned	lhu	I	R[rt]={16'b0,M[R[rs]+SignExtImm](15:0)}	(2)	25_{hex}
Load Linked	ll	I	R[rt] = M[R[rs]+SignExtImm]	(2,7)	30_{hex}
Load Upper Imm.	lui	I	R[rt] = {imm, 16'b0}		f_{hex}
Load Word	lw	I	R[rt] = M[R[rs]+SignExtImm]	(2)	23_{hex}
Nor	nor	R	R[rd] = ~ (R[rs] | R[rt])		0 / 27_{hex}
Or	or	R	R[rd] = R[rs] | R[rt]		0 / 25_{hex}
Or Immediate	ori	I	R[rt] = R[rs] | ZeroExtImm	(3)	d_{hex}
Set Less Than	slt	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0		0 / $2a_{hex}$
Set Less Than Imm.	slti	I	R[rt] = (R[rs] < SignExtImm)? 1 : 0	(2)	a_{hex}
Set Less Than Imm. Unsigned	sltiu	I	R[rt] = (R[rs] < SignExtImm) ? 1 : 0	(2,6)	b_{hex}
Set Less Than Unsig.	sltu	R	R[rd] = (R[rs] < R[rt]) ? 1 : 0	(6)	0 / $2b_{hex}$
Shift Left Logical	sll	R	R[rd] = R[rt] << shamt		0 / 00_{hex}
Shift Right Logical	srl	R	R[rd] = R[rt] >> shamt		0 / 02_{hex}
Store Byte	sb	I	M[R[rs]+SignExtImm](7:0) = R[rt](7:0)	(2)	28_{hex}
Store Conditional	sc	I	M[R[rs]+SignExtImm] = R[rt]; R[rt] = (atomic) ? 1 : 0	(2,7)	38_{hex}
Store Halfword	sh	I	M[R[rs]+SignExtImm](15:0) = R[rt](15:0)	(2)	29_{hex}
Store Word	sw	I	M[R[rs]+SignExtImm] = R[rt]	(2)	$2b_{hex}$
Subtract	sub	R	R[rd] = R[rs] - R[rt]	(1)	0 / 22_{hex}
Subtract Unsigned	subu	R	R[rd] = R[rs] - R[rt]		0 / 23_{hex}

Byte Order in Memory

How to number bytes within a word?

- Little-endian: byte numbers start at the little (least significant) end



- Big-endian: byte numbers start at the big (most significant) end



It doesn't matter which addressing type is used as long as loads and stores are consistent.

Exercise: data.s